

Report Outline

Detecting Market Manipulation in High-Frequency LOB Data

A detailed outline for the final-project written report, structured around the five-part flow of `final_merged_v2.ipynb` and adapted to standard academic-report conventions.

Target length: $\approx$ 10–14 pages of body text plus appendix.

1. Introduction

(1–1.5 pages)

1.1 Problem statement

Detect rare market manipulation patterns — **pump-and-dump**, **layering**, and **quote stuffing** — in high-frequency limit order book data. Motivate why this matters in real markets: market integrity, fair access, regulatory oversight.

1.2 Dataset overview

Five U.S. equity symbols (AAPL, AMZN, GOOG, INTC, MSFT). Train (1,070,022 rows) and val (467,939 rows) are **100% normal**; test (289,101 rows) is a mix of normal and anomalous events. This is a **semi-supervised** anomaly detection problem — we model "normal" and score test rows against it.

1.3 Evaluation

Private-leaderboard ROC-AUC. Baseline 0.88. Submission format: (**ID**, **TARGET**) where **TARGET** is the anomaly probability in [0, 1].

1.4 Approach roadmap

Pipeline summary in one sentence per stage:

1. Clean and harmonize the three splits.
 2. Six-angle EDA to understand what "normal" looks like.
 3. Engineer a 37-feature representation.
 4. Train four progressively-richer simple models (LR / RF / SVM / Autoencoder).
 5. Escalate to advanced unsupervised detectors (Isolation Forest, z-score).
 6. Combine the strongest detectors into a final ensemble.
-

2. Data Preparation

(0.5–1 page)

2.1 Initial inspection

Shapes, dtypes, and column inventories across train / val / test. Note structural differences before any cleaning.

2.2 Quality issues identified

Concrete issues uncovered from inspection:

- `ClusterNo` is **100% NaN** in train and val, and absent from test.
- The row-identifier column is named `ID` in train / val but `index` in test.
- `OriginalSequenceNumber` is `int64` in train / val but a dashed string (e.g. `'16120456-000001-000'`) in test, encoding a sub-event burst.
- 37 scattered NaN values appear in test feature columns; no NaNs in train / val.

2.3 Cleaning actions applied

For each issue: the cleaning decision, the justification, and the line of code that implements it.

Issue	Action	Justification
<code>ClusterNo</code> empty	Drop	Carries zero information
<code>index</code> vs <code>ID</code>	Rename <code>index</code> → <code>ID</code>	Uniform identifier across splits
<code>OriginalSequenceNumber</code> dashed in test	Parse <code>base_seq</code> prefix	Enables burst feature engineering
Test NaNs	Fill with 0	Matches "no event" mode in standardized columns

2.4 Building the normal pool

`normal = pd.concat([train, val])` → 1,537,961-row labelled-normal training pool that backs every downstream fit. No test rows ever participate in fitting.

3. Exploratory Data Analysis

(3–4 pages — covers Part 1 of the notebook)

3.1 Univariate distributions

Notebook Section 1.

- **Descriptive statistics per split** — means clustered near 0, stds near 1, but max values reach +30 to +50 σ . Range is consistent across train and val.
- **Categorical and temporal summaries** — five symbols, single trading day in test, multi-year in train and val.
- **Mass-spike detection** — top-5 most common values per column reveal that `TradeBidSize` and `TradeAskSize` each have ~20% of rows piled at a single negative value (the standardized "no event" mode).
- **Dual histogram view** — linear (zoomed to [p1, p99]) and symlog (full range) plotted side-by-side. Explain why both are needed: linear shows the bulk, symlog reveals the heavy positive tail.

3.2 Train vs. val vs. test comparison

Notebook Section 2.

- **Side-by-side summary statistics** — feature-by-feature mean, std, median across the three splits with delta columns.
- **Kolmogorov–Smirnov tests** — D-statistic for every numeric feature, train↔val and train↔test. Headline finding: train $\equiv$ val ($D < 0.05$); train vs. test diverges, with `TradeBidSize` at **D = 0.65**.
- **Interpretation grid** for the D statistic at our sample size ($< 0.02 \approx$ identical; $> 0.20 \approx$ substantial drift).

3.3 Per-symbol heterogeneity

Notebook Section 3.

- **Symbol distribution differs sharply across splits.** Train / val are roughly balanced (each symbol 17–22%); test is dominated by **INTC (35.8%) + MSFT (33.6%) = 70%** of test.
- **Per-symbol mean / std tables** — **BidSize1** is the standout heterogeneous feature with a **std ratio of 3.15×** (AAPL = 0.38, GOOG = 1.19); every other feature has std ratios under 1.5.

3.4 Joint feature structure

Notebook Section 5.

- **Correlation heatmaps** — Pearson and Spearman, on a 200K-row train sample. Strong bid-side / ask-side block structure; differences between Pearson and Spearman flag non-linear relationships in the heavy-tailed columns.
- **Mahalanobis distance distribution** — empirical distribution vs. the chi-square reference. Fat tails above the diagonal → train is **not** joint-Gaussian → a Mahalanobis baseline will under-perform.

3.5 Manipulation pattern fingerprints

Notebook Section 6.

Pattern	Human description	Columns that react	Visual probe in notebook
Pump-and-dump	Coordinated price excursion followed by reversal	ReturnBid/Ask1 , DerivativeReturn* , Trade*Size	Cumulative-return time series
Layering	Large displayed orders cancelled before they trade	BidSize1 , Cancelled*Size , Trade*Size	(TradeBidSize, CancelledBidSize) scatter
Quote stuffing	Bursts of events with no trades	Inter-event Δt , Trade*Indicator	Train vs. test Δt overlay

3.6 EDA synthesis

Pull the threads from 3.1–3.5 together and state the three high-level findings that drive feature engineering and model choice:

1. The data has been pre-processed onto a near-centered, near-unit scale with heavy positive tails.
2. Test differs from train along multiple confounded axes (date, symbol mix, intraday shape) simultaneously.
3. The three manipulation types live in three different feature families — per-row, joint, and sequential.

4. Feature Engineering

(1 page — covers Part 2 of the notebook)

4.1 Burst features

From the parsed **base_seq** column: **burst_size** = count of rows sharing the same base sequence, **burst_log** = $\log_{1p}(\text{burst_size})$, plus threshold flags **burst_10_49** and **burst_50plus**. Why these are the strongest per-row anomaly signal: test rows with the **'-000001-000'** sub-event encoding share a base sequence and indicate event bursts.

4.2 Per-symbol z-scores

$|x - \mu_{\text{sym}}| / \sigma_{\text{sym}}$ per feature, with $(\mu_{\text{sym}}, \sigma_{\text{sym}})$ fit on the normal pool only. Plus aggregates: `max_zscore`, `mean_zscore`. Per-symbol because `BidSize1` is heterogeneous across symbols (Section 3.3 finding).

4.3 Magnitude and spread features

`max_abs_feat`, `abs_RetBid1`, `abs_RetAsk1`, `return_spread = |RetBid1 - RetAsk1|`. The spread feature is motivated by Section 3.4's finding that returns on the two sides normally co-move; a divergence is a stress / manipulation cue.

4.4 The 37-feature set

Summary table of all 37 features, grouped by family (14 raw + 14 per-symbol z-scores + 2 z-score aggregates + 4 magnitude/spread + 3 burst).

4.5 Scaling

`RobustScaler` (median + IQR) fit on the normal pool only, applied to both normal-pool and test feature matrices. Median/IQR is chosen over mean/std because of the heavy-tail and mass-spike findings in Section 3.1.

5. Simple Supervised Models

(2 pages — covers Part 3 of the notebook)

5.1 The semi-supervised challenge

Train and val are 100% normal — there is no positive class for a supervised classifier to learn from. Explain the workaround and its trade-off.

5.2 Synthetic anomaly injection

Adopted from the validate-AUROC procedure in `v2_dense_autoencoder.ipynb`: sample 20K rows from train and multiply return-related features by a random factor in `[10, 50]` to mimic the magnitude-amplification fingerprint of pump-and-dump. Combine with 100K normal rows from train to build a 120K-row labelled set.

Caveat to flag: this only mimics one manipulation fingerprint (return-magnitude perturbation), so the supervised models trained on these labels will be biased toward catching that pattern.

5.3 Model 1 — Logistic Regression

Linear baseline. Architecture (input → sigmoid). Training setup (`RobustScaler`, 500 max iterations). Top coefficients table. Synthetic AUROC and Kaggle score.

5.4 Model 2 — Random Forest

Non-linear, captures interactions. Setup (100 trees, `max_depth = 15`, `min_samples_leaf = 20`). Top feature importances. Synthetic AUROC and Kaggle score. Discussion of why RF flags far fewer rows than LR — better selectivity.

5.5 Model 3 — Linear SVM

Different inductive bias from RF (max-margin separator). Why `LinearSVC` rather than RBF (scalability). Decision-function range and min-max normalization choice. Top coefficients. Synthetic AUROC and Kaggle score.

5.6 Model 4 — Dense Autoencoder

First **unsupervised** model. Architecture $37 \rightarrow 20 \rightarrow 8 \rightarrow 20 \rightarrow 37$. Training setup (Adam, learning rate 0.001, batch 1024, 50 epochs). Final reconstruction loss. Reconstruction error as anomaly score. Synthetic AUROC and Kaggle score.

5.7 Initial Kaggle results

Tabulate the per-model leaderboard scores from individual submissions. Rank them. Highlight that the Autoencoder beats the supervised models because real anomalies don't all match the return-magnitude synthetic fingerprint.

5.8 First ensemble (RF + AE)

Combine the strongest supervised model with the unsupervised one. Equal-weighted average. **Kaggle score: 0.88376**. Discuss why it didn't clearly beat the AE alone — the RF's bias toward synthetic-anomaly patterns adds noise on rows where the synthetic and real anomaly distributions differ.

6. Advanced Anomaly Detectors

(2 pages — covers Part 5 and the Z-score section of the notebook)

6.1 Isolation Forest

- **Algorithm intuition.** Random partitioning isolates anomalies in fewer splits than normal points.
- **Mathematical formulation.** Expected path length $c(n) = 2 H(n-1) - 2(n-1)/n$ for a binary search tree with n external nodes (where H is the harmonic number). Anomaly score $s(x, n) = 2^{(-E[h(x)] / c(n))}$, where $E[h(x)]$ is the expected isolation depth. Interpretation: $s \approx 1$ means strongly anomalous; $s \approx 0.5$ means normal-ish.
- **Implementation.** Hyperparameters (`n_estimators`, `max_samples`, `contamination`). Why we use it: tree-based, handles mixed scales, robust to mass spikes.
- **Result on Kaggle.**

6.2 Z-score detector

Statistical / rule-based detector. Linear combination of `max_zscore`, `mean_zscore`, `burst_10_49`, `burst_50plus`, `burst_log`, and `abs_RetBid1`, with weights chosen from the EDA conclusions. Why we go back to a simple statistical method after deep models: interpretability, and Section 3 showed the strongest signals are concentrated in a few engineered features.

6.3 Two-model ensemble (IF + AE)

Combine the two strongest unsupervised detectors. Equal weights. **Kaggle score: 0.95379**. Discuss the jump over the supervised ensemble — both components are unbiased toward synthetic anomalies, so they generalize better.

6.4 Final 3-way ensemble (Z-score + IF + AE)

Two weighting strategies presented in the notebook:

- **Strategy A — grid-searched weights.** Purely empirical. Search procedure, best weights found, leaderboard score.
- **Strategy B — EDA-justified weights.** Weights chosen from interpretability and the relative strength of each detector on the manipulation patterns in Section 3.5.

Compare the two strategies and state the final choice.

7. Results and Discussion

(1.5 pages)

7.1 Leaderboard progression

Table or bar chart showing the Kaggle leaderboard score at each milestone, from baseline through the final ensemble.

Model / Ensemble	Kaggle ROC-AUC
Baseline (provided)	0.88
Logistic Regression	...
Random Forest	...
SVM (linear)	...
Autoencoder	...
RF + AE ensemble	0.88376
IF + AE ensemble	0.95379
Final 3-way ensemble	...

7.2 What worked and why

Tie back to the EDA findings:

- Unsupervised detectors outperformed supervised ones because real anomalies don't all match the return-magnitude synthetic fingerprint.
- The Z-score detector helped because Section 3 showed the strongest signals concentrate in a few engineered features.
- Ensembling components with different inductive biases (tree-based isolation vs. neural reconstruction vs. rule-based scoring) gave a clear leaderboard jump.

7.3 Limitations

- Synthetic anomalies only mimic the pump-and-dump fingerprint; layering and quote-stuffing detection rely entirely on engineered features and unsupervised models.
- `BidSize1` per-symbol heterogeneity was only partially addressed via per-symbol z-scores.
- Train spans 8 years (June 21 of 2012-2019) but test is a single day; cross-year drift is a confounder.

7.4 Reproducibility

- Environment: `conda env fe-course`.
- All random seeds fixed (42).
- Notebook structure self-documenting; each cell is rerunnable from a clean state given the three CSVs.
- `requirements.txt` lists `scikit-learn`, `pandas`, `numpy`, `matplotlib`, `scipy`.

8. Conclusion

(0.5 page)

Brief recap of the trajectory:

1. EDA revealed that test differs from train along several confounded axes.
2. Supervised models trained on synthetic anomalies hit a ceiling because real anomalies are more diverse than the synthetic fingerprint.
3. Switching to unsupervised detectors and combining them with a rule-based z-score gave the best leaderboard performance.

Future work in one paragraph: windowed temporal features, per-symbol modelling for **BidSize1**, gradient-boosted ensemble of all detectors, and time-conditioned anomaly thresholds.

Appendix

- **A. Full feature dictionary** — 37 rows, three columns (name, family, brief description).
 - **B. Hyperparameter table** — every model with its hyperparameters in one place.
 - **C. Reproducibility instructions** — exact commands to recreate the environment, run the notebook, and regenerate submission CSVs.
 - **D. References** — Liu, Ting, Zhou (2008) *Isolation Forest*; any other anomaly-detection or LOB-modelling papers consulted.
-

End of outline.